Recoverability-Gated Deliberation for Release-Aware Slow Reasoning in Closed-Loop Driving

Anonymous Authors

***Abstract*—Large language models provide deliberative reasoning for autonomous driving, but inference latency separates the query state from the state in which a response becomes available. Although asynchronous systems preserve fast control during inference, a delayed proposal must still be evaluated against the current traffic state and control command. This paper presents Recoverability-Gated Deliberation (RGD), a two-stage framework for selective slow-reasoner admission and release validation. At query time, RGD admits a request only when delay feasibility, action support, recovery-cost advantage, and scene demand jointly indicate a useful deliberation opportunity. At release, the returned proposal is remapped in the observed state and authorized only after current-state, action-distinctness, and recovery-cost checks. We further introduce Recoverable Value of Deliberation (R-VoD), an offline matched-rollout diagnostic that measures the corrective opportunity retained at release. Closed-loop experiments evaluate paired allocation policies, controlled delays, component effects, matched release-state interventions, reasoner substitution, and simulator adapters. The results show that RGD substantially reduces slow-path requests relative to continual and random invocation while preserving paired task outcomes. Matched rollouts further show that release validation retains the corrective intervention while filtering harmful or negligible delayed proposals.**

***Index Terms*—Autonomous driving, closed-loop decision making, inference latency, large language models, recoverability, selective computation.**

I. Introduction

LARGE language models (LLMs) and vision–language models (VLMs) have been used for semantic scene interpretation, driving-knowledge retrieval, and high-level decision making in autonomous driving systems [1]–[8]. These models provide semantic context and long-horizon intent beyond reactive policies. Closed-loop deployment, however, introduces a release-time mismatch: a reasoning request is issued from one traffic observation, whereas its proposal becomes available in a later state. A high-frequency controller can maintain vehicle control while a deliberative model runs asynchronously, but the system must still determine whether the delayed proposal should influence the current command.

Consider a closing lead vehicle and an initially open adjacent lane. The system may request a lane-change recommendation from the slow reasoner. During inference, traffic evolves and the fast controller continues to act. When the response arrives, the target gap may have narrowed, the original maneuver may no longer be admissible, or the fast controller may already select the same or a lower-cost action. The release decision must test the proposal against the observed state and the contemporaneous fast action.

Maintaining a fast control stream does not resolve this release decision. A returned action may pass an isolated state check yet coincide with the fast action or offer no recovery-cost advantage after traffic evolves. This comparison requires a common action schema, mapper, and state-dependent evaluator. This motivates RGD's separation of query admission from release validation.

Recent studies address relevant parts of this process. Invocation policies use uncertainty, scene conditions, or policy discrepancy to decide when to engage a slow branch [9]–[11]. VLM-based attention allocation prioritizes surrounding vehicles under behavioral uncertainty [12]. AsyncDriver and NetRoller preserve fast control during model inference [8], [13], while DualDrive aligns semantic intent with a real-time policy [14]. Runtime-assurance methods determine whether a candidate action may enter the actuation path [15]. These studies address invocation timing, control continuity, or candidate safety. They do not explicitly compare a delayed proposal with the contemporaneous fast action at release through a shared action interface.

This paper asks whether the evolved traffic state still admits a corrective action and whether the returned proposal provides that action at release. Two constraints guide the design: the fast controller must remain active throughout inference, and rejected proposals must not disrupt the control loop. Both branch outputs are evaluated at the same release state through a shared action mapper and feasibility interface.

RGD treats the slow reasoner as a proposal generator rather than a replacement for the fast controller. The fast action remains the default, and a slow proposal reaches actuation only through the shared mapper and safety interface. This separation permits backend substitution without changing release validation and makes the source of each executed action explicit.

A release state is *recoverable* when the remaining maneuver window admits at least one feasible effective alternative that remains distinct from the fast effective action and exceeds a matched-rollout utility margin. At query time, RGD applies online delay, support, cost, and demand conditions before issuing a slow request. *Actionability* is evaluated at release: the returned mapped command must pass the release-state checks, differ from the mapped fast command, and satisfy the recovery-cost margin. Recoverability describes the release state, whereas actionability describes the returned proposal before final safety projection.

Recoverable Value of Deliberation (R-VoD) quantifies recoverability offline. It compares alternative and fast first actions through matched rollouts from the same logged release state under a common continuation policy. It measures the corrective opportunity remaining at release rather than the utility of a

returned slow proposal. R-VoD is a reward-defined diagnostic, not an online value estimate, and does not enter admission or release validation.

Subject to resource availability, the online gate requires scene demand, delay feasibility, action support, and cost advantage simultaneously. High urgency alone cannot compensate for an expired maneuver window or the absence of a supported alternative. The fast controller supplies the default command at each control cycle during inference. When the response arrives, release validation evaluates the proposal in the current traffic state and accepts it only if it passes current-state mapping, state checks, distinctness from the fast command, and recovery-cost checks.

We evaluate RGD under a shared fast controller, slow reasoner, request budget, and action interface. A paired common-protocol study measures allocation and task outcomes across six policies, with effect sizes, confidence intervals, and multiplicity-adjusted tests. Controlled-delay traces link each request to release validation and final projection. A fixed-query replay isolates release validation on proposals naturally issued by RGD, and matched rollouts compare each distinct intervention with the contemporaneous fast action. Complementary component studies assess admission selectivity, while backend substitution, traffic sweeps, and adapter experiments examine the common action interface.

The main contributions are as follows:

1) We formalize delayed deliberation through two release-time notions: recoverability of the current state and actionability of a returned proposal. We define R-VoD as an offline matched-rollout diagnostic of recoverability under a common continuation policy.
2) We develop RGD with noncompensatory query admission and a release contract aligned with the fast action. The fast controller remains active during inference, and a delayed proposal can affect control only after both branches are mapped and evaluated in the observed release state through the shared safety interface.
3) We evaluate RGD through paired allocation, controlled-delay, component, matched-intervention, backend-substitution, and adapter-transfer studies. The results quantify request efficiency and show that release validation retains the corrective delayed action while filtering harmful or negligible alternatives on a fixed natural query stream.

II. Related Work

A. Language-Conditioned and Generative Driving

Language-conditioned driving methods differ in the proximity of their outputs to vehicle actuation. At the advisory level, DiLu retrieves relevant experience before producing an interpretable decision [1], and Driving with LLMs represents traffic through object-level vectors and returns actions with explanations [16]. VLM-based systems combine visual observations with language reasoning for navigation and motion planning [6], [17], while knowledge retrieval methods supply domain-specific context before decision making [7]. These systems use language-conditioned interfaces to encode scene relations and retrieve driving knowledge.

A second group places model outputs closer to the control boundary. Policy transfer methods use language models to support vehicle decision making [18], GPT-Driver casts planning as language modeling [5], and C-TRAIL integrates commonsense reasoning with trajectory planning [19]. Generative driving methods learn predictive scene representations or future trajectories for downstream planning [20]–[23]. As outputs approach actuation, a delayed proposal may have fewer downstream opportunities for correction and should be evaluated in the state in which it will execute.

These studies generally focus on model capability or driving representation. RGD instead addresses whether a delayed high-level proposal may replace the current fast action. It operates at the interface between a slow proposal generator and the fast controller, without introducing a new language model or trajectory decoder. The release rule evaluates the returned proposal in the state in which it may be executed.

B. Dual Process Reasoning and Selective Computation

Dual-process driving systems divide responsibility between a responsive policy and a deliberative model. LeapAD uses reflection and accumulated experience, LeapVAD adds cognitive perception, and FASIONAD coordinates both branches through adaptive feedback [9], [24], [25]. DualDrive aligns semantic intent with a real-time policy, and NetRoller connects general and specialized models through asynchronous execution [8], [14]. In each design, the fast branch supplies immediate control while the slow branch contributes information or actions at selected intervals.

Selective computation methods determine when the slow branch should run. AdaDrive and AdaThinkDrive adapt deliberation depth to scene demand [10], [26], and G-VLM calibrates invocation from the discrepancy between lightweight and multimodal policies [11]. Behavioral Attention uses a VLM to allocate attention among surrounding vehicles under behavioral uncertainty [12]. These invocation signals are evaluated before inference begins, when neither the returned action nor the release state is known. RGD supplements invocation selection with release-time evaluation against the contemporaneous fast action.

Query-time selection and release-time validation serve different purposes. The first determines whether slow computation should begin; the second determines whether its output remains useful after the state has changed. RGD retains both decisions explicitly rather than treating a successful request as sufficient for release.

Metareasoning balances expected decision improvement against computational cost. Time-bounded and anytime methods schedule reasoning under a finite budget [27]–[30], and concurrent planning permits an executor to continue acting while a planner computes [31]. In driving, an estimate made at query time can differ from the opportunity available at release because traffic evolves while the fast controller continues to act. R-VoD operationalizes the remaining corrective opportunity through matched rollouts.

C. Runtime Safety, Delay, and Viability

Timing mechanisms preserve control during inference. AsyncDriver separates language-model planning from real-time control [13], NetRoller keeps a specialized driving model active [8], and event-triggered or adaptive-computing strategies regulate update frequency [32], [33]. Delay-aware control further models state evolution during input lag [34]. At the assurance layer, Simplex-based systems revert to a trusted controller when a learned action fails an acceptance test [15], and formal viability methods characterize admissible motion under dynamic constraints [35], [36].

These approaches address timing, invocation, or candidate safety. They do not generally formulate a comparison between a delayed proposal and the contemporaneous fast action as a separate release-time decision. RGD complements rather than replaces runtime safety mechanisms. The shared safety interface checks whether a mapped action can enter actuation, while release validation additionally tests whether the delayed proposal is distinct from, and has a recovery-cost advantage over, the contemporaneous fast action. A safe but equivalent proposal leaves the fast action unchanged. The final safety shield remains active after selection. Thus, selective computation supplies query authority, asynchronous execution preserves fast-path continuity, and runtime assurance supplies the final projection; RGD adds an explicit release-time authority test in the evolved state.

III. Recoverability-Gated Deliberation

RGD separates a slow-reasoner call into query admission and release validation. Admission determines whether the scene warrants deliberation and whether a feasible alternative is supported under the expected delay. Release validation checks the returned proposal in the observed release state. The fast controller supplies the default command while inference is pending and whenever either stage rejects the slow path.

A release state s_r is *recoverable* if it admits a feasible effective alternative that remains distinct from the fast effective action and satisfies the fixed R-VoD diagnostic criterion. R-VoD measures this state-level opportunity offline. A returned proposal is *actionable at release* only if its mapped command passes the contemporaneous state checks, differs from the mapped fast command, and satisfies the proposal-specific recovery-cost margin. The final safety projection may still map an authorized command to the same executed action as the projected fast command. R-VoD and release validation serve complementary rather than interchangeable roles.

Fig. 1 depicts the two online gates and the offline branch that constructs R-VoD labels from matched first-action rollouts. The fast controller remains active at every cycle. Driving memory supplies episodic context to the slow reasoner after admission, while both gates are evaluated from the current control state and the shared action interface.

The architecture preserves fast-path control during slow reasoning. A slow request never suspends the fast controller, and a missing response, mapping failure, or rejected release retains the fast command. The reasoner is a proposal source rather than a second controller. This boundary keeps inference latency and response-format errors outside the control cadence.

The release comparison is action-aligned. Both branch outputs first enter the same mapped command space, after which the same feasibility and cost interfaces are applied. Semantically equivalent outputs map to the same maneuver, independent of backend wording. The shared final safety shield remains the last stage for either source.

A. Query and Release Process

At frame t, the ego vehicle observes state s_t. The fast policy produces an action that the shared mapper converts to a_t^f. Both branches use this mapper and the same final safety projection $\Phi(s, a)$, which maps a command to an executable action in state s. Each branch output is converted into a maneuver and target speed; different expressions are equivalent if they induce the same mapped behavior. The high-level action domain comprises lane-left, lane-hold, lane-right, accelerate, and decelerate, all mapped through the same simulator interface. Specifically, the response parser $\kappa(y)$ normalizes a structured action identifier or canonical action token, and $\mathcal{M}(s, y) = \kappa(y)$ only when $\kappa(y) \in \mathcal{A}(s)$; otherwise, $\mathcal{M}$ is undefined. This operation is current-state semantic mapping, not trajectory replanning.

The returned proposal is mapped in the observed release state s_r. A lane-change or speed command available at query time may no longer be valid; a mapping or feasibility failure leaves a_r^f selected.

The delay estimate is used only for admission. The actual release frame r is the first control frame at which the response is available, and returned and fast actions are evaluated in the observed state s_r. Responses unavailable before episode termination are not released.

B. Offline Measure of Corrective Opportunity

Offline matched rollouts quantify the corrective opportunity at release. At each logged release snapshot s, let $a^f_{\text{eff}} = \Phi(s, a^f)$ denote the effective fast first action after the shared mapper and final safety projection. $\mathcal{D}(s)$ contains feasible effective first actions that remain distinct from a^f_{eff} after the same stages, and $\mathcal{C}(s) = \{a^f_{\text{eff}}\} \cup \mathcal{D}(s)$. Let $J_H(s, a)$ denote the matched-rollout utility. For horizon H,

$$J_H(s,a) = \frac{\sum_{k=0}^{H-1} \gamma^k r_k}{\sum_{k=0}^{H-1} \gamma^k} - \mathbb{I}[\text{collision within } H].$$

We use $H = 20$ and $\gamma = 0.99$. All currently legal commands pass through the same safety projection and actuator bridge, and candidates with the same discrete command and target speed are merged. The Recoverable Value of Deliberation (R-VoD) is

$$\text{R-VoD}(s) = \max_{a \in \mathcal{C}(s)} \left[J_H(s,a) - J_H(s, a^f_{\text{eff}}) \right]. \tag{1}$$

Only the first effective action is changed; the fast policy supplies the continuation. Because $a^f_{\text{eff}} \in \mathcal{C}(s)$, R-VoD is nonnegative and equals zero when no alternative improves the matched fast continuation. A state is labeled corrective when R-VoD is at least 0.02.

The matched design fixes the logged release state, traffic realization, finite horizon, discounted reward definition, continuation policy, and termination rule. Accordingly, $J_H(s, a)$ is

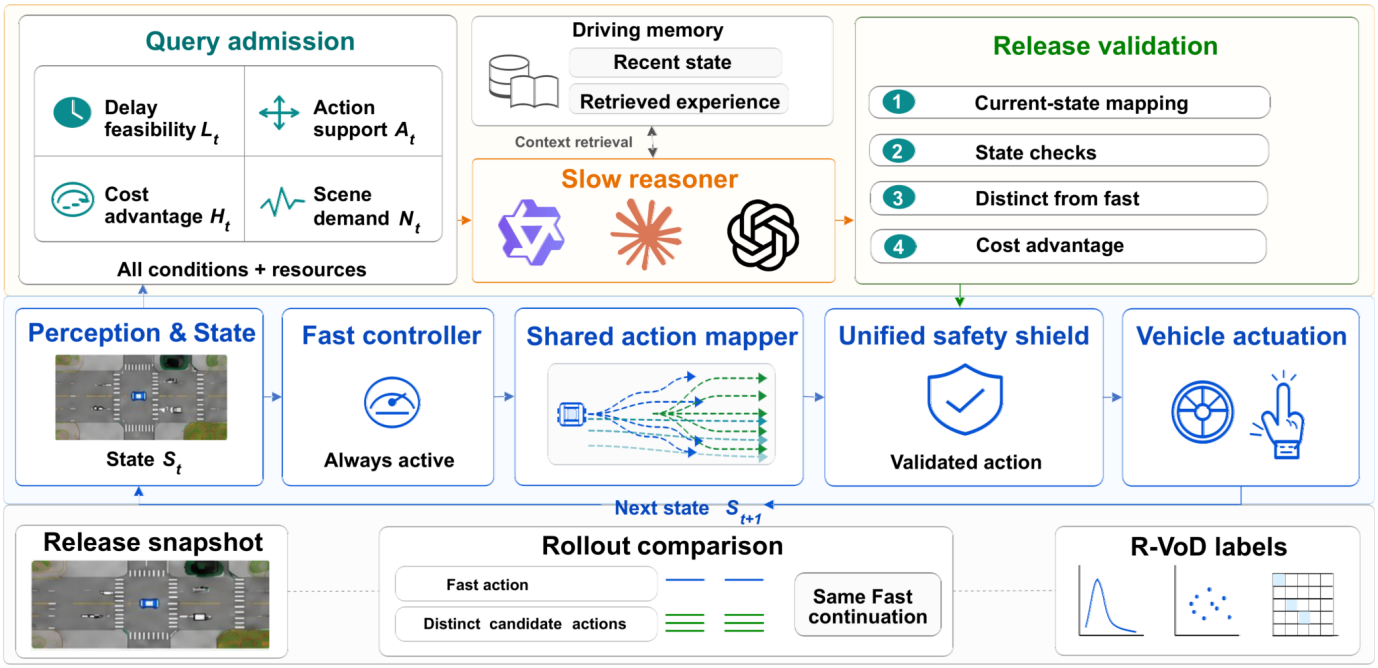


Fig. 1. Overview of the RGD architecture and online/offline information flow.

an offline, reward-defined utility for the diagnostic and is never queried by the online gate. By contrast, $c_t(a)$ is a runtime recovery-cost score used to rank current actions at admission and to test a returned command at release. The two quantities support different decisions and are not interchangeable.

Changing only the first effective action isolates the corrective opportunity available in the release state under the common simulator continuation. R-VoD provides a response-independent offline label for evaluating admission. Release validation tests whether a returned proposal satisfies the online release contract, and the final projection determines its executed effect.

C. Query Admission

Let $\mathcal{A}(s_t)$ be the exact runtime action domain and let a_t^f denote the mapped fast command. The common one-step evaluator defines

$$c_t(a) = \mathrm{clip}_{[0,1]}\left(h_t + \left[d_t(a) - \min_{b\in\mathcal{A}(s_t)} d_t(b)\right]_+\right),$$

where d_t combines the RSS/DCBF safety decomposition with comfort and efficiency terms, and h_t is the maximum contemporaneous TTC, headway, longitudinal, lateral, and interaction pressure. Missing or nonfinite costs and inconsistent action domains fail closed. The permanent guard F_t requires complete domain agreement and a nonempty set $\mathcal{D}_t = \{a \neq a_t^f : c_t(a) \le \kappa\}$.

The four admission scores have direct runtime definitions. With policy frequency f, predicted latency $\widehat{\tau}_t$, critical maneuver window T_t^{crit}, and reserve ρ, the residual-window score is $\ell_t = [(T_t^{\mathrm{crit}} - \lceil f\widehat{\tau}_t\rceil/f - \rho)/T_t^{\mathrm{crit}}]_{[0,1]}$. For the feasible alternative families $\mathcal{F}_t \subseteq \mathcal{F}$, let u_m be the minimum support cost in family m and $u_* = \min_m u_m$. The four tests are

$$L_t = \mathbb{I}[\ell_t \ge \lambda_L],$$
$$A_t = \mathbb{I}\left[|\mathcal{F}|^{-1}\sum_{m\in\mathcal{F}_t} e^{-(u_m-u_*)/T_A} \ge \lambda_A\right],$$
$$H_t = \mathbb{I}\left[\kappa^{-1}[c_t(a_t^f) - \min_{a\in\mathcal{D}_t} c_t(a)]_+ \ge \lambda_H\right],$$
$$N_t = \mathbb{I}[\max(h_t, n_t^{\mathrm{pre}}) \ge \lambda_N].$$

We fix $\kappa = 0.55$, $T_A = 0.10$, and $(\lambda_L, \lambda_A, \lambda_H, \lambda_N) = (0.05, 0.55, 0.10, 0.20)$. The conjunction is noncompensatory.

Resource availability, including the request budget, cooldown, and executor status, is represented by $v_t \in \{0,1\}$; $v_t = 1$ also requires that no slow request is pending. The admission rule is

$$q_t = \mathbb{I}[v_t \wedge F_t \wedge L_t \wedge A_t \wedge H_t \wedge N_t], \tag{2}$$

where $\mathbb{I}[\cdot]$ is the indicator function.

If latency evidence, the action-cost vector, or the slow executor is unavailable, admission is suppressed and the fast controller continues without issuing a slow request.

D. Release Validation

At the actual release frame r, let y_r be the returned response and $\mathcal{M}$ the shared current-state mapper. The fast policy produces an action that the mapper converts to a_r^f from the same state. If $\mathcal{M}(s_r, y_r)$ is undefined, the response is rejected and the fast command is retained. Otherwise, let $\widetilde{a}_r^{\mathrm{sl}} = \mathcal{M}(s_r, y_r)$ denote the mapped slow command. Domain integrity, action support, and recovery headroom are recomputed in s_r; delay feasibility and scene demand remain admission-only tests. Thus, N_t determines whether computation is initiated, whereas release authority is determined

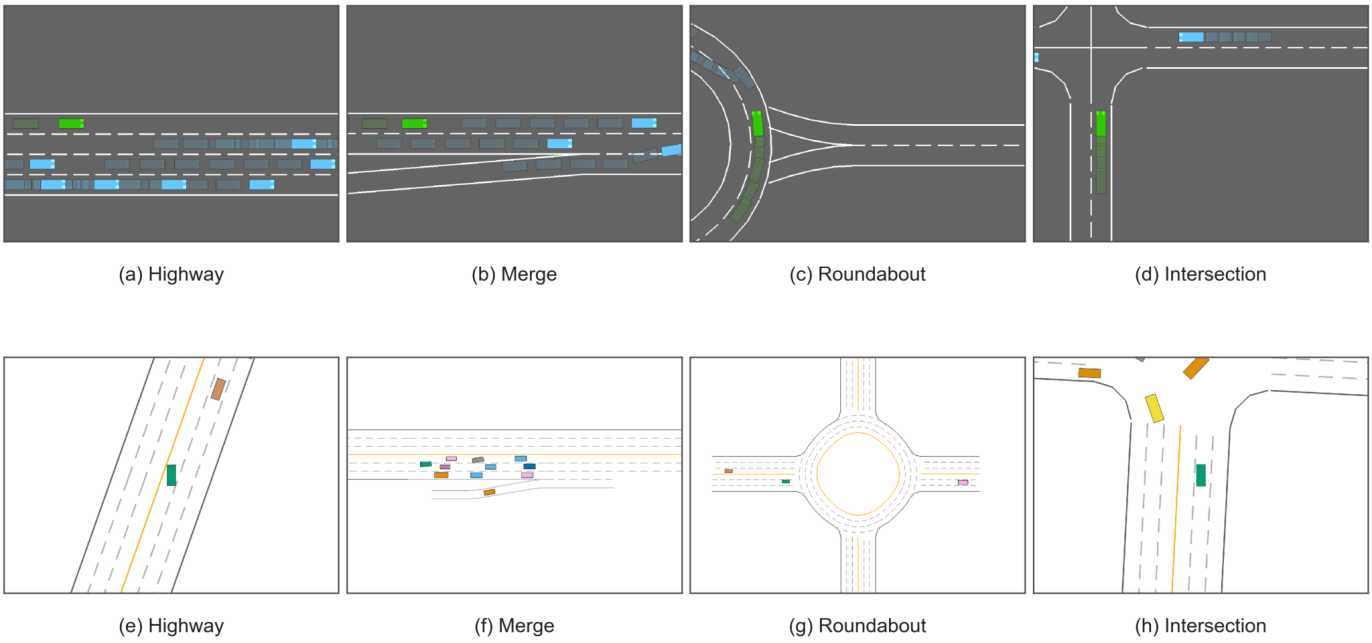


Fig. 2. Representative highway-env and MetaDrive scenes.

by current command feasibility and proposal-specific advantage. Let $\mathcal{V}_r = \{a \in \mathcal{A}(s_r) : F_r \wedge A_r \wedge H_r \wedge c_r(a) \leq \kappa\}$ contain the current-state-feasible mapped commands that pass the recomputed checks. Here c_r is the same recovery-cost evaluator at s_r, with lower values preferred. The release decision in Fig. 1 is

$$g_r = \mathbb{I}\big[\widetilde{a}_r^{\mathrm{sl}} \in \mathcal{V}_r \wedge \widetilde{a}_r^{\mathrm{sl}} \neq a_r^f \wedge c_r(\widetilde{a}_r^{\mathrm{sl}}) + \delta \leq c_r(a_r^f)\big]. \quad (3)$$

Here, the release margin is fixed at $\delta = 0.02$.

The selection and execution stages are therefore

$$a_r^{\mathrm{sel}} = \begin{cases} \widetilde{a}_r^{\mathrm{sl}}, & g_r = 1, \\ a_r^f, & g_r = 0, \end{cases} \qquad a_r^{\mathrm{exec}} = \Phi(s_r, a_r^{\mathrm{sel}}). \quad (4)$$

Equation (4) gives the fast-path non-interference property directly: if a response fails the release contract, then $a_r^{\mathrm{exec}} = \Phi(s_r, a_r^f)$. Distinctness is tested between mapped commands before Φ, so an authorized slow command may still be projected to the same executable action as the fast command. We report this post-projection outcome separately. The release-state H condition establishes that a lower-cost alternative exists, whereas the final term in (3) tests whether the returned command meets the required margin. No R-VoD label enters the online decision.

Release validation extends feasibility with distinctness and recovery-cost comparison. A proposal admitted at query time receives no persistent authority; all response-specific checks use the observed release state. With the proposal stream fixed, this contract predicts that retained actions should provide positive matched utility over the contemporaneous fast action, whereas unvalidated execution need not do so.

IV. Experiments and Discussion

A. Experimental Setup

We evaluate RGD on the highway-env [37] and MetaDrive [38] closed-loop simulators with Qwen3-8B [39] as the default slow reasoner. The main highway-env experiments operate at 10 Hz for 30 s. Within each comparison, all methods share the seed cohort, action schema, fast controller, slow-reasoner interface, request budget, and evaluation rule. Gate development, main evaluation, and mechanism analysis use disjoint cohorts. The paired allocation study uses 30 seeds and six policies, yielding 180 policy–seed units. Each component study uses a separate 20-seed cohort; the complete six-arm audit contains 120 arm–seed units.

The simulator seed is the experimental and bootstrap unit. We report 95% percentile bootstrap intervals from 20,000 seed resamples. Binary paired endpoints use exact McNemar tests, continuous endpoints use paired sign-flip tests, and the five prespecified RGD–baseline contrasts are Holm-adjusted within each endpoint. R-VoD is computed only after release through matched first-action rollouts. Gate criteria, corrective margins, and the service-latency stratum are selected on the development or calibration cohorts and remain fixed throughout the reported evaluation.

Each analysis targets a distinct system property. System-level context places the closed-loop implementation in established task settings. The delay sweep tests whether admission allocates slow computation according to the available maneuver window, while the response trace tests the release contract after the traffic state evolves. The component study examines how selected admission conditions shape the states sent to the slow path. Backend substitution tests the common action interface under different reasoners, and the traffic and simulator studies characterize the fixed interface across evaluated road structures and adapters.

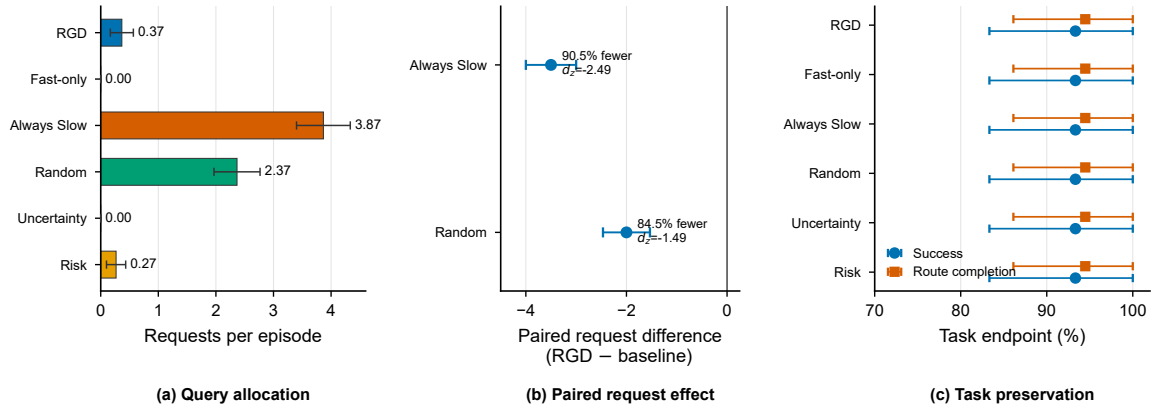


Fig. 3. Paired common-protocol allocation over 30 seeds. (a) Mean requests with 95% seed-bootstrap intervals, (b) paired RGD-minus-baseline request differences, and (c) success and route-completion estimates. p_{H} denotes the Holm-adjusted paired sign-flip result. RGD reduces request burden relative to continual and random invocation while preserving matched task endpoints.

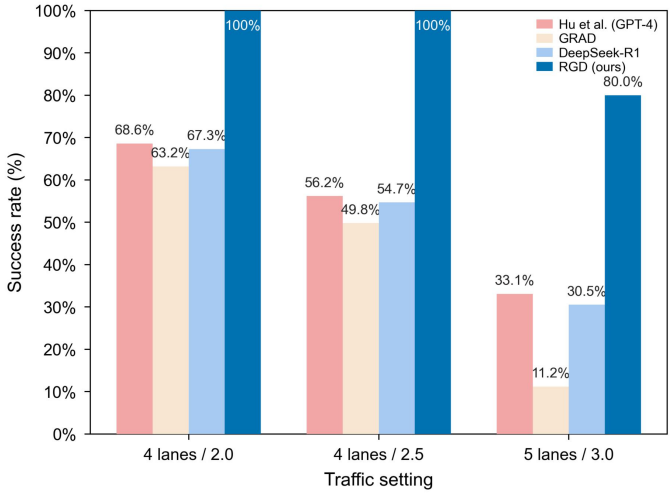


Fig. 4. Success-rate context under the highway-env settings of Hu et al.

The gate is selected on seeds 2000–2039 across 0.7-, 1.7-, and 2.7-s delay strata, with at most six requests per seed–delay cell and a 21-frame minimum request spacing. Candidate tuples first maximize the minimum Full-versus- leave-one-out corrective-state margin, then minimize requests per corrective state, and finally prefer the stricter tuple. The resulting criteria are frozen before the go/no-go, main, and mechanism cohorts; no evaluation outcome retunes the gate.

The paired allocation cohort provides the primary resource and task-endpoint comparison. Separate controlled-delay cohorts contain more request and release events, enabling event-level analysis of timing, validation, and projection. To isolate release validation, a matched audit freezes the 11 proposals naturally issued by RGD on the 30 paired seeds. The release-validated and no-validation arms receive the same proposals and release timing. Every post-projection distinct action is then compared with Fast from its authenticated online release snapshot using the $H = 20$ matched rollout defined in Sec. III.

Fig. 2 provides representative highway-env and MetaDrive states for the road geometries used in the evaluation. The upper row shows highway, merge, roundabout, and intersection scenes; the lower row shows the corresponding MetaDrive reconstructions. The paired layouts show the road geometries in which the common high-level action interface is evaluated.

TABLE I
SYSTEM-LEVEL CONTEXT UNDER THE PADRIVER SETTING.

Method	Dist.	Speed	Saf.	Kep.	Comp.	Time
Driving with LLMs	411	49.42	0.49	0.56	20	48.0
GPT-Driver	428	51.40	0.45	0.52	23	36.2
DiLu	386	46.29	0.60	0.72	28	24.1
PADriver–Normal	603	72.47	0.91	0.92	25	1.4
RGD (ours)	611	73.60	0.75	0.86	29	0.0376

B. System-Level Context

Under the PADriver experimental conditions [40], we report RGD values alongside source-reported results. Table I places the system within the operating ranges reported under the PADriver setting. RGD completes 29 of 30 runs, records 611 m and 73.60 km/h, and operates at 37.6 ms per frame without a dedicated training phase on the evaluation environment. The fast controller remains active at every frame, whereas the slow reasoner is invoked selectively. Saf. and Kep. denote safe-distance keeping and lane keeping, respectively.

Hu et al. [6] define success as 30 consecutive collision-free frames. We adopt their three traffic settings and ten seeds per setting with no added delay. Fig. 4 reports success rates alongside the published values. Table II provides RGD's operating metrics under the settings stated in the source paper.

Under the settings stated by Hu et al., RGD attains 100% success at 4L/2.0 and 4L/2.5. At 5L/3.0, it attains 80% success and schedules 0.8 slow-path requests per episode. Table II reports safety-qualified speed, assigns zero to unsuccessful episodes, and averages requests over the ten episodes in each setting. The per-frame runtime remains below 66 ms in all three settings.

C. Delay-Controlled Allocation and Component Evidence

1) Paired Common-Protocol Allocation: The paired allocation study compares RGD with Fast-only, Always Slow,

TABLE II
RGD OPERATING METRICS UNDER THE SETTINGS OF HU ET AL.

Setting	Dist. (m)	Speed (m/s)	Req./ep.	Time (ms/frame)
4L / 2.0	69.29	23.84	1.0	58.9
4L / 2.5	66.19	22.76	1.0	65.2
5L / 3.0	52.38	17.38	0.8	50.1

Random, Uncertainty, and Risk scheduling under a common 30-seed protocol. The methods use the same Qwen3-8B service, fast controller, action mapper, release interface, and episode realization; only the slow-path allocation policy differs. Fig. 3(a) reports requests per episode, and panel (b) reports the paired RGD-minus-baseline effects for the two request-intensive allocators.

RGD averages 0.37 requests per episode, reducing slow-path use by 90.5% relative to Always Slow and by 84.5% relative to Random. The paired differences are -3.50 requests [95% CI -4.00, -3.00] and -2.00 requests [95% CI -2.47, -1.53], with standardized effects $d_z = -2.49$ and -1.49, respectively. Both contrasts remain significant after Holm correction ($p_{\mathrm{H}} = 0.00025$). Fast-only and Uncertainty provide the zero-request resource anchors without engaging the slow reasoner.

The six policies produce identical seed-wise success, collision, route completion, reward, distance, and speed outcomes. As shown in Fig. 3(c), each policy completes 28 of 30 episodes and attains a mean route completion of 94.5%. RGD runs at 28.78 ms per frame, well within the 100-ms control interval. Its 11 requests yield nine returned proposals. The result establishes RGD's resource advantage over continual and random deliberation while preserving the underlying fast-path behavior under the shared protocol. Release authority is evaluated separately on the same frozen natural proposal stream.

2) Zero Added Delay Reference: The zero-delay cohort fixes the fast controller, slow reasoner, request budget, and feasibility interface across all six allocation policies; only the request and release rules differ. The delay sweep then uses the same seeds and opportunity schedule to evaluate delay-feasibility effects on slow-path allocation.

Without added delay, Fig. 5(a)–(b) shows that RGD schedules fewer slow-path requests and has a lower per-frame runtime than Always Slow. This indicates that query admission reduces continual slow-path use under the same fast controller and slow reasoner.

Across the delay sweep in Fig. 5(c), distance confidence intervals overlap while request counts decrease as delay increases. This pattern is consistent with greater selectivity under the delay-feasibility condition. The fixed gate adjusts its invocation rate without per-delay retuning.

3) Allocation With a 1.7-s Added Delay: The 1.7-s experiment examines how RGD processes a slow response after the traffic state has evolved. The fast controller, slow reasoner, request budget, seed cohort, and feasibility interface remain fixed. Fig. 6 reports the response-validation stages and the resulting release decisions.

Table III reports a separate 30-seed backend evaluation under the 1.7-s setting. "Calls" denotes successful service responses rather than request attempts in the main cohort.

The controlled delay places each returned proposal in a state later than the state that initiated the request. RGD applies a returned proposal only after all release checks.

4) Response Validation and Component Effects: Response traces follow each request from admission through response validation, current-state mapping, and actuation. At the same 1.7-s added delay, the leave-one-out variants use a separate 20-seed cohort; each removes one admission condition while retaining the release procedure.

Under the 1.7-s delay, 143 RGD requests return valid responses and 138 reach release before episode termination. Current-state checks admit 28 responses to release comparison; nine mapped slow commands differ from the contemporaneous fast command, and six satisfy the recovery-cost margin. After the shared final projection Φ, four of these authorized commands match the projected fast command and two remain distinct. Fig. 6(a) reports the complete event funnel, while panel (b) summarizes all 28 state-valid outcomes. A delayed proposal can affect the final command only after it satisfies the release contract and remains distinct after projection; otherwise, the fast action remains in control.

Fig. 7(a)–(b) shows that removing L, A, or H changes both the number of requests and the additional release states relative to Full RGD. Panel (c) reports the ablation-minus-Full-RGD difference in R-VoD-labeled corrective-state rate with 95% seed-bootstrap confidence intervals. The H contrast has a confidence interval entirely above zero, directly linking the online cost-advantage condition to the corrective-state composition of the selected cohort. The request and additional-release-state changes in panels (a)–(b) further show that L and A shape which states are sent to the slow path. Together, these analyses establish the complementary roles of delay feasibility, action support, and recovery-cost advantage in the admission rule.

The corrective-state rate in Fig. 7(c) is labeled offline by R-VoD, whereas H_t is computed online from the recovery-cost evaluator. The two quantities serve different roles: H_t tests for a current cost advantage before a request is issued, whereas R-VoD labels the opportunity that remains at the observed release state through matched rollouts. The H contrast links an online admission signal with a separately constructed release-state diagnostic.

5) Complete Gate and Projection Audit: We complement the admission study with a six-arm paired audit covering Full RGD, removal of each of L, A, H, and N, and joint removal of H and N. All arms replay the same gate-independent proposal source over 20 held-out seeds at a fixed 1.7-s delay. The action mapper, release rule, safety projection, budget, and cooldown remain unchanged, so each contrast removes only the named admission predicate.

The shared bank is frozen from a Qwen3-8B Always-Slow source that does not read the evaluated gate. Each record binds the source frame, raw action, response outcome, measured latency, and response hash; all arms replay the same bank and recompute the Fast action at every frame. Gate decisions are completed before proposal fields are exposed to an arm.

Fig. 8(a) traces the 70 synchronized candidates through the serial admission stages. Full RGD retains 49 candidates after

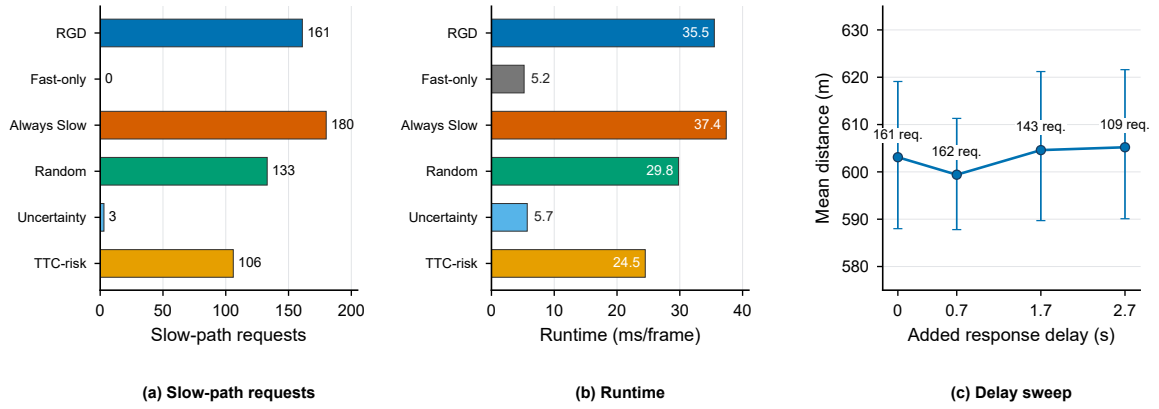


Fig. 5. Zero-delay comparison and delay sweep. (a) Slow-path request totals, (b) mean runtime per frame, and (c) mean distance with 95% seed intervals; labels in panel (c) give request counts.

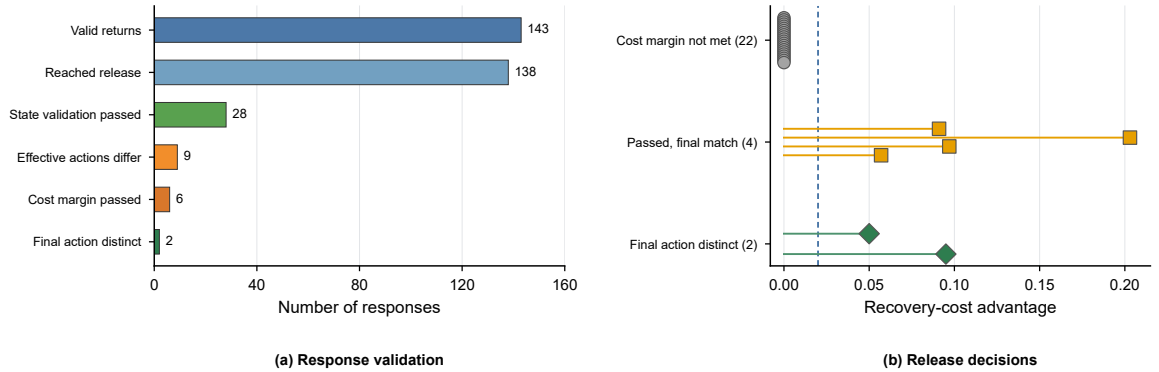


Fig. 6. Response validation under delayed feedback. (a) Response funnel and (b) release-decision outcomes by recovery-cost advantage.

TABLE III
RESULTS WITH DIFFERENT SLOW REASONERS.

Reasoner	Calls	Distance (m)	Completion
Qwen3-8B	138	599.16	0.97
Qwen3.5-4B	138	599.16	0.97
Qwen2.5-7B	122	599.12	0.97
GPT-5.6-sol	112	603.39	0.97
GPT-5.6-terra	132	599.16	0.97
GPT-5.6-luna	133	599.16	0.97
Fable-5	137	599.16	0.97

TABLE IV
RESULTS ACROSS LANE AND DENSITY SETTINGS.

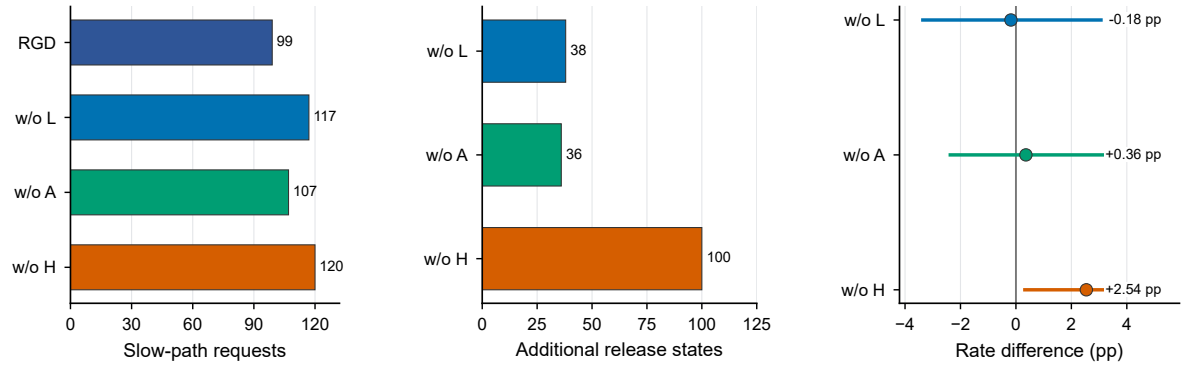
Lanes/Density	Distance (m)	Successful Runs (/30)
4L/2.0	599	29
4L/3.0	478	23
5L/2.0	629	30
5L/3.0	429	20
6L/2.0	611	28
6L/3.0	453	20

TABLE V
SUCCESS-RATE CONTEXT ACROSS MANEUVER AND TRAFFIC-DENSITY SETTINGS.

	Traffic Scenario					Left Turn Traffic Density		
Method	Left	Right	Straight	Merge	Round.	1.0	2.0	3.0
VLM attention allocation [12]	0.966	0.991	0.984	0.987	0.974	0.975	0.961	0.948
RGD (ours)	0.842	0.908	0.868	0.986	0.747	0.906	0.863	0.801

L, 40 after A, eight after H, and none after N. Removing L retains 70 candidates at the first stage and 61 after A, whereas removing A retains 49 after the second stage. These trajectories expose the distinct upstream screening roles of L and A. Removing H, N, and H/N leaves 1, 8, and 40 candidates after the complete gate, respectively. Panel (b) follows these three arms through the slow-path lifecycle: 1, 8, and 40 requests are issued, of which 1, 5, and 26 reach delayed release. The stagewise audit therefore separates candidate reduction by L/A from the final workload control provided by H/N.

The command-level trace separates release exposure from executable effect. Removing N and H/N yields 4 and 16 mapped proposals distinct from the fast command. The shared final projection maps all of them to the fast-equivalent executable action, and task endpoints remain invariant across all



(a) Slow-path requests **(b) Additional release states** **(c) Corrective-state rate**

Fig. 7. Effects of the admission components. (a) Requests, (b) additional release states, and (c) corrective-state-rate differences (ablation minus Full RGD) with 95% seed-bootstrap intervals. The H ablation produces the largest increase in corrective-state rate, whereas L and A primarily change the request and release-state exposure.

120 paired arm–seed units. This audit identifies the dominant $H \times N$ interaction in slow-path workload and verifies non-interference at the execution boundary.

The fixed-query audit then replays the 11 natural RGD proposals through the current release contract. Both release-validated and no-validation arms obtain nine releases. Without validation, three returned commands remain distinct after the common projection; release validation retains one. Fig. 8(c) reports their matched utility. The retained action improves J_H from 0.667 to 0.721, an 8.2% relative gain, and increases 20-step progress by 23.70 m without a collision in either branch. The two filtered actions have utility advantages of -0.0325 and 0.0047, both below the fixed 0.02 corrective margin. Thus, the release contract preserves the only corrective intervention in the fixed proposal stream and removes the harmful and negligible alternatives.

D. Reasoner-Interface Evaluation

We replace only the slow reasoner while retaining the RGD gate, action schema, and fallback path. Table III reports closely aligned distance and completion values across the seven evaluated backends, from local models to API services. Successful service responses range from 112 to 138, whereas mean distance ranges from 599 to 603 m and every backend attains a completion rate of 0.97. All backends use the same action mapping, current-state validation, and recovery-cost comparison. Under this shared interface, the system-level point estimates remain closely aligned across the evaluated backends.

E. Evaluation Across Traffic and Simulators

1) Traffic Structure: The auxiliary sweep includes the default 4L/2.0 setting and five lane/density variants without retuning the gate. Table IV reports mean distance and successful runs over 30 episodes per setting. At density 2.0, successful runs range from 28 to 30 and mean distance from 599 to 629 m. Across all six cells, the fixed configuration records 20–30 successful runs per setting. These results describe operation of the same interface under the evaluated traffic structures.

TABLE VI
COMPOUND-HAZARD CONTEXT IN METADRIVE.

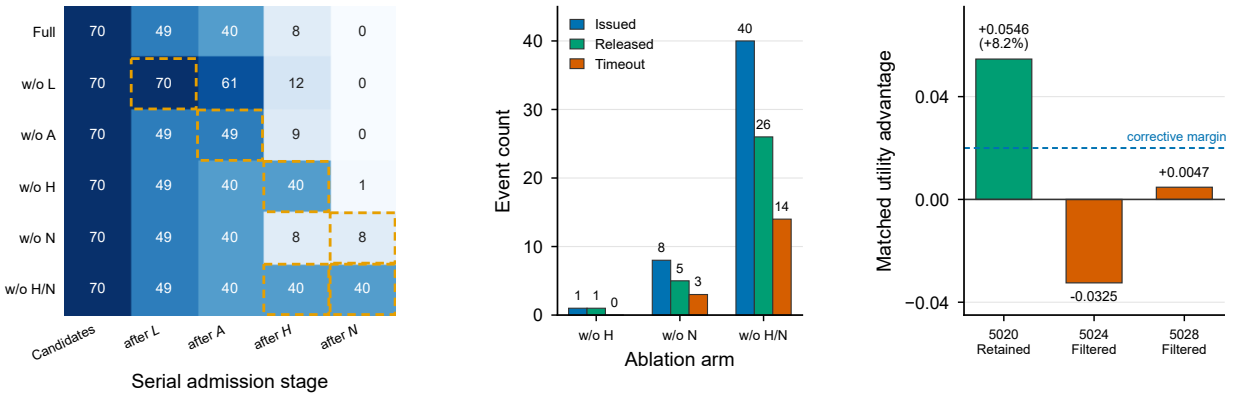
Method	Scene	Safety (%)	Speed (m/s)
ASR-RL [18]	A	99.2	7.1
	B	98.3	8.6
	C	97.9	6.8
RGD (ours)	A	100.0	4.8
	B	94.8	7.0
	C	84.6	6.9

2) Simulator Transfer: The MetaDrive experiment introduces simulator-specific observation and action adapters at zero response delay, with no modification to the gate configuration used in highway-env. The shared action schema, admission rule, and fallback order are applied without retuning. This study evaluates transfer of the common action interface across heterogeneous simulator dynamics; delayed release-time behavior is characterized by the highway-env experiments. Fig. 2 shows the distinct road geometries and traffic layouts across the two environments. The shared mapper, admission rule, and fast-path fallback are held fixed across the corresponding evaluation settings. The zero-delay MetaDrive study exercises the common release interface through the simulator adapter. The MetaDrive panels reconstruct the logged ego state and recorded task-relevant neighbors at the selected frame while preserving the traffic configuration used by the evaluated traces.

F. Evaluation on Additional Scenario Families

The published ASR-RL and VLM-attention rows retain their source definitions and evaluation scopes, whereas the RGD rows are measured in this work. Together, the values provide task-level context across the reported scenario families.

1) Compound Hazard Scenes: Following the experimental conditions stated by ASR-RL [18], the MetaDrive evaluation covers pedestrian crossing (A), uncontrolled intersection (B), and combined vehicle-pedestrian traffic (C), each with 1,000 episodes. Table VI reports the RGD results alongside source-reported ASR-RL context. RGD attains 100% collision-free



(a) Stagewise retention **(b) Slow-path lifecycle** **(c) Release validation**

Fig. 8. Complete admission and release audit. (a) Serial candidate retention and (b) slow-path lifecycle over the 20 paired component seeds; dashed cells mark omitted predicates. (c) Matched utility advantage for the three distinct interventions in the fixed natural query stream; the dashed line is the corrective margin.

safety in Scene A and applies the same gate configuration across the three compound-hazard scenes without policy-library training or scenario-specific retuning.

2) Maneuver and Density Changes: Following the settings stated in the source paper, five maneuver classes and a left-turn density sweep are evaluated at 2 Hz for 30 s with 1,000 episodes per cell. Table V reports the RGD success rates alongside published VLM attention-allocation values [12] as task-level context. RGD achieves a success rate of 0.986 on merge, and the same high-level action interface and gate logic are applied across the five evaluated maneuver scenarios.

G. Discussion

RGD converts slow reasoning from a persistent command source into a release-conditioned proposal. Before computation, admission determines whether a supported lower-cost alternative can remain useful within the expected response interval. After the state evolves, release validation compares the returned proposal with the action selected by the fast controller in the same state. The final safety projection then determines the executable command. This sequence distinguishes recoverability of the release state, actionability of the proposal, and its post-projection effect.

The paired common-protocol study isolates the principal system-level advantage. RGD uses substantially fewer slow-path requests than continual and random invocation, with large paired effects and unchanged seed-wise task outcomes. The zero-request methods provide the resource floor, while RGD retains access to deliberation when all four admission conditions hold. Thus, RGD occupies a selective operating point between disabling slow reasoning and invoking it throughout the episode.

The controlled-delay studies characterize the admission decision, and the response trace evaluates the release contract. The delay sweep shows that the fixed admission rule changes slow-path allocation as the response interval changes. The trace follows each proposal through current-state mapping, recomputed state checks, mapped-command distinctness, recovery-cost comparison, and the shared safety projection. The fixed-query matched replay further shows that the release contract retains the corrective intervention and rejects the harmful and negligible alternatives. Together with Eq. (4), these results connect release authorization to executed action value while preserving fast-path fallback.

The component analyses provide two complementary views of the gate. The R-VoD study connects L, A, and H to the composition of selected release states. The complete six-arm audit further shows that N, reinforced by H, controls release exposure and mapped-command disagreement. The matched release audit closes the mechanism chain from admission to executed value. Backend substitution, the traffic sweep, and cross-simulator studies address a separate system property: the common action interface operates with different reasoner outputs, road structures, and simulator adapters under the evaluated settings.

Scope and limitations. This study considers discrete high-level commands, controlled added delays, a fixed fast-policy continuation for R-VoD, and the evaluated simulator adapters; MetaDrive evaluates interface transfer at zero response delay. Continuous trajectory proposals and broader service-latency profiles remain for future evaluation.

V. Conclusion

This paper presents Recoverability-Gated Deliberation, a two-stage framework that determines when to invoke slow reasoning and whether a delayed LLM proposal may enter the control loop. RGD admits a request only when the joint delay, support, cost, and demand conditions hold. At release, it authorizes a mapped proposal only when it passes the current-state checks, differs from the contemporaneous fast command, and satisfies the recovery-cost margin. R-VoD provides an offline matched-rollout diagnostic of the corrective opportunity available at release.

Paired experiments show that RGD sharply reduces slow-path use relative to continual and random invocation while preserving task outcomes. Controlled-delay traces and complete component analysis verify the distinct roles of admission, release validation, and final projection. Matched release-state rollouts show that validation retains the corrective delayed

action while filtering harmful or negligible alternatives. The common interface is further exercised across LLM backends, traffic structures, and simulator adapters. By treating slow reasoning as an expiring proposal, RGD establishes a selective, release-aware decision layer between low-rate deliberation and high-frequency control.

REFERENCES

[1] L. Wen *et al.*, "DiLu: A knowledge-driven approach to autonomous driving with large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[2] Z. Xu *et al.*, "DriveGPT4: Interpretable end-to-end autonomous driving via large language model," *IEEE Robot. Autom. Lett.*, vol. 9, no. 10, pp. 8186–8193, 2024.

[3] H. Shao *et al.*, "LMDrive: Closed-loop end-to-end driving with large language models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 120–15 130.

[4] C. Sima *et al.*, "DriveLM: Driving with graph visual question answering," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 256–274.

[5] J. Mao, Y. Qian, J. Ye, H. Zhao, and Y. Wang, "GPT-Driver: Learning to drive with GPT," arXiv preprint arXiv:2310.01415, 2023. [Online]. Available: https://arxiv.org/abs/2310.01415

[6] Y. Hu, D. Ou, J. Huang, M. Wu, M. Hao, and R. Yu, "Integrating vision and language foundation models for enhanced navigation and decision-making in connected autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 74, no. 10, pp. 16 233–16 249, 2025.

[7] D. Pei, J. He, K. Liu, Y. Wu, Y. Lei, and X. Xiao, "Enhancement of large language models driving knowledge for practical autonomous driving decision making," *IEEE Trans. Veh. Technol.*, pp. 1–17, 2026, early access.

[8] R. Xin *et al.*, "NetRoller: Interfacing general and specialized models for end-to-end autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 75, no. 5, pp. 7469–7482, 2026.

[9] K. Qian *et al.*, "FASIONAD: Fast and slow fusion thinking systems for human-like autonomous driving with adaptive feedback," arXiv preprint arXiv:2411.18013, 2024. [Online]. Available: https://arxiv.org/abs/2411.18013

[10] R. Zhang *et al.*, "AdaDrive: Self-adaptive slow-fast system for language-grounded autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 5112–5121.

[11] X. Chen, X. Wang, W. Chen, and J. Gao, "An adaptive multimodal end-to-end autonomous driving framework based on behavior discrepancy," *IEEE Trans. Intell. Transp. Syst.*, pp. 1–12, 2026, early access.

[12] B. Ma, H. Liu, R. Zhong, P. Liu, X. Zhou, and J. Ma, "Behavioral uncertainty-aware attention allocation via VLMs for interactive autonomous driving," *IEEE Trans. Veh. Technol.*, pp. 1–14, 2026, early access.

[13] Y. Chen, Z.-H. Ding, Z. Wang, Y. Wang, L. Zhang, and S. Liu, "Asynchronous large language model enhanced planner for autonomous driving," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 22–38.

[14] L. Li, J. Fang, J. Xue, and C. Lv, "Vision–language model-enabled dual-system for autonomous driving in safety-critical transportation," *IEEE Trans. Intell. Transp. Syst.*, pp. 1–12, 2026, early access.

[15] S. Chen, Y. Sun, D. Li, Q. Wang, Q. Hao, and J. Sifakis, "Runtime safety assurance for learning-enabled control of autonomous driving vehicles," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 8978–8984.

[16] L. Chen *et al.*, "Driving with LLMs: Fusing object-level vector modality for explainable autonomous driving," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 14 093–14 100.

[17] R. Zhao *et al.*, "VLM-Driver: Human-like autonomous driving decision-making via vision language model," *IEEE Trans. Veh. Technol.*, vol. 75, no. 5, pp. 7327–7342, 2026.

[18] J. Wang, H. Ren, X. Zhu, and Z. Ma, "Enhancing autonomous vehicle decision-making through policy transfer with large language model," *IEEE Trans. Intell. Transp. Syst.*, pp. 1–10, 2025, early access.

[19] Z. Cui *et al.*, "C-TRAIL: A commonsense world framework for trajectory planning in autonomous driving," *IEEE Trans. Veh. Technol.*, pp. 1–14, 2026, early access.

[20] X. Wang, Z. Zhu, G. Huang, X. Chen, J. Zhu, and J. Lu, "DriveDreamer: Towards real-world-drive world models for autonomous driving," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 55–72.

[21] C. Min *et al.*, "DriveWorld: 4D pre-trained scene understanding via world models for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 15 522–15 533.

[22] W. Zheng, R. Song, X. Guo, C. Zhang, and L. Chen, "GenAD: Generative end-to-end autonomous driving," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024, pp. 87–104.

[23] B. Liao *et al.*, "DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 12 037–12 047.

[24] J. Mei *et al.*, "Continuously learning, adapting, and improving: A dual-process approach to autonomous driving," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 123 261–123 290.

[25] Y. Ma *et al.*, "LeapVAD: A leap in autonomous driving via cognitive perception and dual-process thinking," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 37, no. 4, pp. 1963–1977, 2026.

[26] Y. Luo *et al.*, "AdaThinkDrive: Adaptive thinking via reinforcement learning for autonomous driving," arXiv preprint arXiv:2509.13769, 2025. [Online]. Available: https://arxiv.org/abs/2509.13769

[27] S. Russell and E. Wefald, "Principles of metareasoning," *Artif. Intell.*, vol. 49, no. 1–3, pp. 361–395, 1991.

[28] M. Boddy and T. L. Dean, "Deliberation scheduling for problem solving in time-constrained environments," *Artif. Intell.*, vol. 67, no. 2, pp. 245–285, 1994.

[29] S. Zilberstein, "Using anytime algorithms in intelligent systems," *AI Mag.*, vol. 17, no. 3, pp. 73–83, 1996.

[30] B. Cserna, W. Ruml, and J. Frank, "Planning time to think: Metareasoning for on-line planning with durative actions," in *Proc. Int. Conf. Autom. Plan. Scheduling (ICAPS)*, vol. 27, 2017, pp. 56–60.

[31] A. Elboher, A. Bensoussan, E. Karpas, W. Ruml, S. S. Shperberg, and E. Shimony, "A formal metareasoning model of concurrent planning and execution," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 10, 2023, pp. 12 427–12 435.

[32] P. Tabuada, "Event-triggered real-time scheduling of stabilizing control tasks," *IEEE Trans. Autom. Control*, vol. 52, no. 9, pp. 1680–1685, 2007.

[33] M. Li, J. Gao, L. Zhao, and X. Shen, "Adaptive computing scheduling for edge-assisted autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5318–5331, 2021.

[34] T. G. Molnar, A. K. Kiss, A. D. Ames, and G. Orosz, "Safety-critical control with input delay in dynamic environment," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 4, pp. 1507–1520, 2023.

[35] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "On a formal model of safe and scalable self-driving cars," arXiv preprint arXiv:1708.06374, 2017. [Online]. Available: https://arxiv.org/abs/1708.06374

[36] A. Liniger and J. Lygeros, "Real-time control for autonomous racing based on viability theory," *IEEE Trans. Control Syst. Technol.*, vol. 27, no. 2, pp. 464–478, 2019.

[37] E. Leurent, "An environment for autonomous driving decision-making," GitHub repository, 2018, version 1.12. [Online]. Available: https://github.com/Farama-Foundation/HighwayEnv

[38] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, "MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3461–3475, 2023.

[39] A. Yang *et al.*, "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[40] G. Kou *et al.*, "PADriver: Towards personalized autonomous driving," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2025, pp. 1–8.